

FINAL PROJECT REPORT

Capstone G1 Shop

Multilingual E-Commerce Storefront

Team: Weijie, Shukla, and Jade

Project start: 21 July 2026

Report revision: 6 August 2026

Repository: [Jade-pang-coder/capstone-project-groupone](https://github.com/Jade-pang-coder/capstone-project-groupone)

Backend API: [Hosted REST API](#)

A current record of implementation, architecture, verification, deployment, limitations, and recommendations.

Executive Summary

Capstone G1 Shop is a responsive e-commerce storefront developed with React 19 and Vite 8. The application lets visitors browse and search a product catalogue, filter products by category, view product details, manage a cart, and complete checkout as either a guest or a registered customer. Registered customers can sign in and review their previous orders. Guests can retain and retrieve recent order information on the same browser by using an order code.

The interface supports English, Simplified Chinese, Malay, and Tamil. It also provides light and dark themes, responsive page layouts, loading and error states, and fallback imagery for products without a usable image.

This repository contains the frontend only. It communicates with a separately deployed REST API for users, products, categories, registered-user carts, orders, and order items. The API implementation and database schema are not included in this repository, so backend internals cannot be verified from this codebase. The current solution is suitable for a capstone demonstration, but the security and transaction limitations in Section 11 must be addressed before production use.

1. Project Background

1.1 Problem Statement

Customers expect an online shop to provide a simple path from product discovery to order confirmation. The project therefore needed to combine catalogue browsing, account access, cart management, checkout, localization, and order history in one consistent interface while integrating with a persistent backend service.

1.2 Project Objectives

The project objectives were to:

- create a clear and responsive online storefront;
- allow customers to shop without mandatory registration;
- support customer registration, login, session persistence, and logout;
- retrieve products and categories from a REST API;
- support product search, category filtering, and product detail views;
- provide guest and registered-customer cart flows;
- create orders and order-item records through the API;
- provide registered-customer order history and same-browser guest order lookup;
- support four interface languages and two appearance themes; and
- produce a deployable optimized frontend with supporting documentation.

1.3 Project Scope

The implemented frontend includes:

- home, product catalogue, and product detail views;
- client-side product search by name or SKU;
- client-side category filtering;
- registration and JWT-based login through the hosted API;
- local session restoration from browser storage;
- guest carts stored in browser local storage;

- API-backed carts for authenticated customers;
- quantity updates, item removal, cart totals, and empty-cart handling;
- guest and registered-customer checkout;
- order confirmation and registered-customer order history;
- same-browser guest order lookup by order code;
- English, Chinese, Malay, and Tamil translations;
- persistent light and dark themes; and
- responsive CSS layouts and product image fallback behavior.

The following are outside the implemented scope:

- real payment authorization or payment gateway integration;
- password reset, email verification, social login, or multi-factor authentication;
- an administration interface;
- product reviews, wish lists, recommendations, coupons, and returns;
- shipping-provider integration, invoices, and email notifications;
- URL-based routing with React Router; and
- automated unit, integration, and end-to-end test suites.

2. Users and Functional Requirements

2.1 User Types

User type	Available capabilities
Guest	Browse, search, filter, view products, use a local cart, check out, view the latest confirmation, and look up locally retained guest orders.
Registered customer	All shopping capabilities, an API-backed cart, profile-based checkout details, account discount when supplied by the API, and personal order history.
Administrator	No administration interface is implemented in this repository. Product and category mutation functions exist in API modules but are not exposed through the UI.

2.2 Primary User Journey

1. The shopper opens the home page and browses featured products or categories.
2. The shopper may register, log in, or continue as a guest.
3. The shopper searches or filters the catalogue and opens a product detail view.
4. The shopper adds one or more products to the cart.
5. The shopper adjusts quantities or removes unwanted items.
6. The shopper enters contact and shipping information at checkout.
7. The application refreshes product data, calculates the order values, creates the order, and then creates its order-item records through the API.
8. The application clears the cart and displays an order confirmation.
9. A registered customer can review the order from My Orders. A guest can retrieve an order retained in the same browser by entering its order code.

2.3 Implemented User Stories

ID	User story	Implementation status
US-01	As a visitor, I can browse products without an account.	Implemented
US-02	As a visitor, I can register and automatically sign in.	Implemented through <code>POST /users</code> , followed by <code>POST /login</code>
US-03	As a customer, I can log in and log out.	Implemented
US-04	As a shopper, I can search by product name or SKU.	Implemented in the browser
US-05	As a shopper, I can filter products by category.	Implemented in the browser
US-06	As a shopper, I can view product details and choose a quantity.	Implemented
US-07	As a shopper, I can add, update, and remove cart items.	Implemented
US-08	As a guest, I can complete checkout without registering.	Implemented
US-09	As a registered customer, I can complete checkout with my order linked to my account.	Implemented
US-10	As a registered customer, I can view my prior orders.	Implemented, with client-side ownership filtering
US-11	As a guest, I can look up my order by its code.	Implemented for orders saved in the same browser only
US-12	As a shopper, I can use the interface in one of four languages.	Implemented
US-13	As a shopper, I can choose light or dark appearance.	Implemented
US-14	As a customer, I can make a real online payment.	Not implemented
US-15	As an administrator, I can manage products in a dedicated interface.	Not implemented

3. Technology Stack

Area	Technology	Role in the project
UI library	React 19	Component-based storefront interface
Build tooling	Vite 8	Development server and optimized production build
Language	JavaScript and JSX	Application and component logic
Styling	Plain CSS	Responsive layouts, component styles, and themes
Localization	i18next and react-i18next	Translation and active-language management
State management	React Context and hooks	Authentication, cart, and theme state
Persistence	Browser local storage	Session, language, theme, guest cart, and guest order cache
Data access	Browser Fetch API	Communication with the hosted REST API
API	Separate hosted REST service	Users, login, products, categories, carts, orders, and order items
Hosting target	Vercel or another static host	Deployment of the generated <code>dist</code> directory

Area	Technology	Role in the project
Version control	Git and GitHub	Team collaboration and source history

`bcryptjs` is listed as a frontend dependency but is not used by the current source code. Password verification is performed by the backend login endpoint.

4. System Architecture

4.1 Logical Architecture

```

User's browser
├── React/Vite single-page application
│   ├── Page components
│   ├── Auth, Cart, and Theme contexts
│   ├── i18next translation resources
│   ├── localStorage
│   └── Fetch-based API modules
│       ├── Hosted REST API on Vercel
│       └── Backend data store (outside this repository)

```

The application entry point wraps `App` with `ThemeProvider`, `AuthProvider`, and `CartProvider`. `App.jsx` stores the active page in React state and renders the matching page component. Page components call functions in `src/api`, while shared user, cart, and theme data are supplied through React Context.

4.2 Navigation Model

The project does not use React Router. Navigation keys such as `products`, `product/5`, and `order-confirmation/12` are held in component state and interpreted by `App.jsx`.

Internal page key	View
<code>home</code>	Home and featured products
<code>products</code>	Product catalogue, search, and filters
<code>product/<id></code>	Product details
<code>cart</code>	Cart contents and totals
<code>login</code>	Customer login
<code>register</code>	Customer registration
<code>checkout</code>	Shipping form and order summary
<code>order-confirmation/<id></code>	Confirmation for a newly created order
<code>dashboard</code>	Protected customer order history
<code>track-order/<code></code>	Same-browser guest order lookup

Because navigation is held in memory, a browser refresh returns the application to the home page and views do not have directly shareable URLs.

4.3 Frontend Structure

src/	
— api/	REST API request and response-normalization modules
— assets/	Imported image assets
— component/	Reusable header, menus, cards, cart items, and controls
— context/	Authentication, cart, and theme providers
— i18n/	English, Chinese, Malay, and Tamil resources
— pages/	Storefront page components and page-specific CSS
— utils/	Product image fallback helper
— App.jsx	Application shell and state-based page selection
— main.jsx	React entry point and provider composition

5. Major Implementation Areas

5.1 Product Catalogue

Products are retrieved from `GET /products` and normalized so that API fields such as `title/name` and `unit_price/price` can be displayed consistently. Categories are retrieved from `GET /categories`.

The home and products pages support category filtering. The products page also performs case-insensitive searches against product names and SKUs. Product detail pages request a single product and its category, provide quantity controls, prevent values below one, and disable ordering when a product is inactive. Invalid images are replaced with a local placeholder.

5.2 Authentication

Registration sends the customer's full name, normalized email, password, default membership tier, and zero discount to `POST /users`. After successful registration, the application calls `POST /login`. Login returns a JWT and user record.

The frontend stores the JWT and user information in local storage and restores them when the application reloads. Protected content checks for both a token and a user object. Logout clears the stored authentication data.

This frontend does not independently verify JWT expiry or refresh tokens. Actual authentication and authorization guarantees therefore depend on the separately deployed backend.

5.3 Cart Management

Cart behavior differs by user type:

- Guest items are maintained in React state and saved to the `cart` local-storage key. A generated guest session token is also retained for compatibility with guest cart APIs, although current guest add/update/remove operations remain local.
- Registered-customer carts are resolved or created through `/carts`. Their cart items are read and changed through `/cart-items`, with Bearer authorization when a token is available.
- Product details are fetched to hydrate registered cart items for display.
- If an authenticated cart request fails, the context attempts to use the local cached cart.

The cart calculates item count and total price in the browser. It supports adding an existing product, changing quantities, removing items, clearing the cart, and blocking checkout when empty.

5.4 Checkout and Orders

Checkout collects the customer's name, email, phone number, and shipping address. For a registered customer, the latest profile is retrieved before order creation and the account email is used.

Before submission, the frontend re-fetches every product and calculates line subtotals using the latest returned prices. It reads the registered customer's `discount_percentage`, calculates the discount, creates a unique order code, and sends an order record to `POST /orders`. It then creates each order item separately through `POST /order-items`.

The payment-method radio buttons are demonstrational. The selected method is not included in the order request, no card details are collected, and no payment provider is contacted.

For registered customers, the confirmation view retrieves the new order and its items from the API. For guests, confirmation data and up to 20 recent guest orders are cached in local storage so they can be displayed again in the same browser.

5.5 Order History and Tracking

The protected dashboard retrieves orders from `GET /orders` and filters them in the browser by the signed-in user's ID. Selecting an owned order retrieves its details and order items.

Guest lookup searches locally stored guest-order records by `order_code` or `order_number`. It does not query a restricted server-side tracking endpoint. Consequently, guest tracking works only on the same browser and device where checkout occurred and only while that local data remains available.

5.6 Localization and Theme

The interface includes translation resources for:

Code	Language
en	English
zh	Simplified Chinese
ms	Malay
ta	Tamil

The selected language is saved under `appLanguage` and restored on startup. English is the fallback. Language changes also update the document language and title. Product catalogue translations are resolved through catalogue helper keys when translations are available.

The theme toggle applies or removes the `dark` class on the document root and stores the selection under `theme`. Theme styling remains usable even if storage is unavailable.

6. REST API Integration

The frontend currently hard-codes this base URL in each API module:

```
https://capstone-project-backend-delta.vercel.app/api
```

6.1 Endpoints Used by the Interface

Method	Endpoint	Frontend use
POST	/login	Authenticate a customer and receive a JWT
POST	/users	Register a customer
GET	/users/:id	Refresh profile data for checkout
GET	/products	Load the catalogue
GET	/products/:id	Load current product details and prices
GET	/categories	Load category filters and category details
GET, POST	/carts	Find or create a registered-customer cart
GET, POST	/cart-items	Load and add registered-customer cart items
PUT, DELETE	/cart-items/:id	Change or remove registered-customer cart items
GET, POST	/orders	Load order history or create an order
GET	/orders/:id	Load one registered-customer order
POST	/order-items	Create an order line
GET	/order-items/order/:id	Load lines for an order

Additional create, update, and delete helpers for products, categories, carts, and orders exist in `src/api`, but they are not called by the current storefront UI.

6.2 Response Normalization

The API modules normalize several possible backend response shapes. Examples include:

- `title` to `name` and `unit_price` to numeric `price` for products;
- `full_name` to `name` for users;
- `order_code` to `order_number` for display;
- `subtotal_amount` and `total_amount` to numeric `subtotal` and `total`; and
- `product_title` to `product_name` for order items.

This compatibility layer reduces coupling between page components and variations in API field naming.

7. Data Model Used by the Frontend

The backend schema is outside this repository. The following model is inferred only from request bodies and response fields used by the frontend.

Entity	Fields used by the frontend
User	<code>id</code> , <code>full_name</code> / <code>name</code> , <code>email</code> , <code>password</code> , <code>membership_tier</code> , <code>discount_percentage</code> , optional address and phone
Category	<code>id</code> , <code>name</code> and display metadata returned by the API

Entity	Fields used by the frontend
Product	<code>id</code> , <code>category_id</code> , <code>sku</code> , <code>title/name</code> , <code>description</code> , <code>unit_price/price</code> , <code>image_url</code> , <code>is_active</code> , <code>created_at</code>
Cart	<code>id</code> , <code>nullable user_id</code> , <code>optional session_token</code>
Cart item	<code>id</code> , <code>cart_id</code> , <code>product_id</code> , <code>quantity</code>
Order	<code>id</code> , <code>nullable user_id</code> , <code>order code</code> , <code>auth token</code> , <code>customer_type</code> , <code>customer contact fields</code> , <code>shipping address</code> , <code>subtotal</code> , <code>discount</code> , <code>total</code> , <code>status</code> , and <code>timestamp</code>
Order item	<code>id</code> , <code>order_id</code> , <code>product_id</code> , <code>sku</code> , <code>product-title snapshot</code> , <code>unit price</code> , <code>quantity</code> , and <code>subtotal</code>

Expected relationships are: category to products, user to carts and orders, cart to cart items, product to cart/order items, and order to order items. Database constraints and transaction behavior must be confirmed in the backend repository.

8. User Interface and Accessibility

The application uses a consistent header containing home and product navigation, authentication controls, language selection, a theme toggle, and a live cart count. The home page provides a hero call to action, category menu, and product grid. Separate layouts support product details, cart management, checkout, confirmation, tracking, and order history.

Accessibility-oriented implementation includes:

- explicit labels for form controls;
- semantic buttons for application navigation;
- alternative text for product images;
- screen-reader text and accessible labels for language and theme controls;
- disabled button states during relevant operations; and
- visible loading, error, empty, and success states.

Manual keyboard, screen-reader, contrast, and cross-browser verification is still required before claiming formal accessibility conformance.

9. Testing and Verification

9.1 Automated Project Checks

The following checks were executed on 5 August 2026:

Check	Result	Notes
<code>npm run lint</code>	Passed with warnings	No ESLint errors; four <code>react-hooks/exhaustive-deps</code> warnings
<code>npm run build</code>	Passed	Vite 8.1.5 transformed 92 modules and generated the production assets

The four lint warnings concern missing effect dependencies in `DashboardPage.jsx`, `OrderConfirmationPage.jsx`, `ProductDetailsPage.jsx`, and `ProductsPage.jsx`. They do not block the build but should be resolved to avoid stale closures and unclear effect behavior.

9.2 Manual Acceptance Coverage

The detailed manual test catalogue is maintained in [docs/TEST_CASES.md](#). It covers:

- installation, linting, building, and preview startup;
- home-page and navigation behavior;
- catalogue display, search, filtering, details, quantity limits, and image fallback;
- registration, login, logout, and protected content;
- guest and registered carts;
- checkout, confirmation, dashboard history, and guest lookup;
- language and theme persistence;
- responsive layouts and accessibility basics;
- API failure behavior; and
- production smoke testing.

No automated test runner is configured in `package.json`, and this report does not claim that every manual case has been executed. Test execution should record the tester, date, build or commit, environment, result, actual behavior, and evidence.

9.3 Recommended Demonstration Flow

1. Open the home page and switch language and theme.
2. Filter a category and search for a product by name or SKU.
3. Open product details and add a chosen quantity to the cart.
4. Change the quantity and show the recalculated cart total.
5. Complete a guest checkout and save the generated order code.
6. Use Track Order to retrieve the locally retained guest order.
7. Register or log in, add a product, and complete a customer checkout.
8. Open My Orders and display the newly created order and its items.

10. Deployment

The frontend builds to static files in `dist` and can be hosted on Vercel or another HTTPS static host. The required build settings are:

Setting	Value
Install command	<code>npm ci</code>
Build command	<code>npm run build</code>
Output directory	<code>dist</code>
Supported Node.js	20.19+ or 22.12+
Runtime frontend secrets	None

The deployed frontend must be permitted by the backend's CORS configuration. The current repository does not record a confirmed production frontend URL. Deployment, smoke testing, rollback, and troubleshooting instructions are in [docs/DEPLOYMENT.md](#).

11. Limitations and Risk Assessment

Priority	Limitation or risk	Impact	Recommended improvement
Critical	Prices, discounts, and totals are calculated in the browser and submitted to the API.	A modified client could submit incorrect financial values if the backend trusts them.	Make the backend accept product IDs and quantities only, then validate stock and calculate all totals server-side.
Critical	The order is created before order items, and each item is posted separately.	A failure can leave an order with missing or partial lines.	Provide one transactional checkout endpoint that creates the order and all items atomically.
High	Dashboard orders, carts, and cart items are retrieved broadly and filtered in the browser.	Data exposure is possible if the backend does not enforce ownership.	Add authenticated, ownership-scoped endpoints such as <code>/orders/my-orders</code> and <code>/carts/me</code> ; enforce access on every item request.
High	JWT and user data are stored in local storage without client-side expiry handling.	Tokens are exposed to successful XSS and stale sessions can remain displayed.	Use strong server validation, short token lifetimes, safe refresh/session design, CSP, and explicit 401 handling.
High	Guest tracking is local-storage lookup rather than secure server tracking.	Orders cannot be tracked on another device and disappear when browser data is cleared.	Add a rate-limited server lookup requiring an order code plus a second verifier.
High	Payment options are display-only.	The interface could imply a payment was processed when none occurred.	Label checkout as demonstration-only or integrate a compliant hosted payment flow.
Medium	The API base URL is repeated and hard-coded in every API module.	Changing environments requires source edits and rebuilding.	Centralize it with <code>import.meta.env.VITE_API_BASE_URL</code> .
Medium	Navigation uses transient component state.	Refreshes return home; views cannot be bookmarked or shared.	Adopt React Router and configure static-host fallback routing.
Medium	There are four hook dependency warnings.	Effects may use stale values after future changes.	Stabilize callbacks with <code>useCallback</code> or define async work inside each effect.
Medium	No automated tests are configured.	Regressions rely on manual discovery.	Add component/unit tests and browser-based end-to-end coverage for the purchase journey.
Low	<code>bencryptjs</code> is an unused frontend dependency.	Adds unnecessary dependency surface.	Remove it unless a justified frontend use is introduced.

12. Project Outcome and Recommendations

The project delivers the principal capstone storefront journey: product discovery, customer access, cart management, checkout, order confirmation, order history, localization, and theming. The production build completes successfully, and the code is organized into pages, reusable components, contexts, translation resources, and API modules.

The solution is demonstration-ready after the hosted frontend and backend are smoke-tested together. It should not be presented as production-ready for real customers or payments until server-side ownership controls, transactional checkout, server-calculated prices and discounts, secure guest lookup, and stronger session handling are verified or implemented.

Recommended next steps, in priority order, are:

1. replace the multi-request checkout with one server-side transactional endpoint;
2. enforce server-side price, discount, stock, and resource-ownership rules;
3. add scoped order/cart endpoints and secure guest order retrieval;
4. centralize environment configuration and introduce URL routing;
5. resolve the four lint warnings;
6. add automated tests for authentication, cart, checkout, and order history; and
7. record the final frontend URL, deployed commit, manual test evidence, and presentation screenshots.

13. Conclusion

Capstone G1 Shop demonstrates a coherent React e-commerce frontend integrated with a hosted API. Its strongest outcomes are the complete guest and customer shopping flows, practical response normalization, multilingual presentation, theme persistence, responsive interface, and clear separation of shared state and API access. The documented limitations provide a realistic path from a successful educational prototype to a more secure and maintainable production architecture.

References

- [README.md](#) — project overview, setup, structure, and limitations
- [docs/DEPLOYMENT.md](#) — deployment and operational guide
- [docs/TEST_CASES.md](#) — manual acceptance test catalogue
- [package.json](#) — dependency and command definitions
- [src/App.jsx](#) — navigation and application shell
- [src/context](#) — authentication, cart, and theme state
- [src/api](#) — REST integration modules
- [src/i18n](#) — language configuration and translation resources
- Scope Document.pdf - official MVP objectives, scope, assumptions, constraints, deliverables, success criteria, and five-day timeline (21 July 2026)
- FED Project.docx - consolidated planning record, wireframes, ER diagram, repository and deployment records, presentation outline, and submission checklist

- Frontend source review - /Volumes/D/Capstone-project-groupone/capstone-project-groupone/capstone-project-groupone, branch doc_update, commit bce2dd5 (reviewed 6 August 2026)

Appendix A - Scope Traceability and Delivery Evidence

This appendix reconciles the official Scope Document planning record with the implementation described in this final report. Status labels distinguish implemented frontend behavior, evidence recorded in the supplied documents, and items that were not independently verified during this document review.

A.1 Assumptions and Constraints

Area	Scope baseline	Project interpretation
Team and prerequisite knowledge	Teams of 3-5 learners who have completed the React, Node.js, Express, PostgreSQL, Git, and deployment modules.	The named team has three members. The project uses the technologies expected by the course, although the backend remains a separate codebase.
Schedule	Development is limited to five working days.	A stable product-to-order vertical flow takes priority over optional retail features.
Connectivity and hosting	Internet access is available; frontend and backend are deployed to Vercel and PostgreSQL is hosted on Neon.	The FED Project records the relevant hosted URLs. Live service and database verification were outside this document-only review.
Product media	Product images may use hosted URLs instead of file uploads.	The frontend consumes image URLs and provides fallback imagery when an image is unusable.
Validation and interface depth	Only essential validation is required, with functionality prioritized over advanced UI design.	The implementation adds responsive states, localization, and themes, while formal accessibility and cross-browser testing remain outstanding.
Excluded integration	Third-party payment processing is excluded.	Checkout records orders but does not authorize or capture payment.

A.2 Scope and Success-Criteria Traceability

Requirement	Evidence in the final report	Assessment
Registration, login, logout, JWT session, and protected dashboard	Sections 2.3 and 5.2 describe registration, login, local session restoration, logout, and protected customer content.	Implemented in the frontend; backend authentication guarantees remain dependent on the separate API.
Products retrieved from persistent data	Sections 5.1 and 6.1 document catalogue, detail, and category requests to the hosted REST API.	Frontend integration is implemented; PostgreSQL persistence is not independently visible from the frontend repository.
Shopping cart	Sections 5.3 and 2.3 cover guest local carts and API-backed registered-customer carts.	Implemented, including quantity changes, removal, totals, persistence behavior, and empty-cart prevention.
Orders stored successfully	Sections 5.4, 5.5, and 6.1 document order and order-item requests, confirmation, and history retrieval.	Implemented client flow; the report correctly flags the non-transactional multi-request checkout risk.
Frontend communicates with backend APIs	The hosted API base URL and endpoint catalogue are recorded in Section 6.	Implemented through Fetch-based API modules.
Application deployed	The FED Project records deployed client, backend API, and administrator URLs.	Deployment evidence is recorded

Requirement	Evidence in the final report	Assessment
Required documentation submitted	README, deployment guide, test catalogue, report, wireframes, and ER diagram are referenced or included.	Evidenced.
React Router technical scope	Section 4.2 documents state-based navigation and Section 11 records the limitation.	Scope variance: React Router is not implemented, so refresh and shareable URL behavior remain limited.
Optional administration	Project records a separate administration repository and deployment.	Optional companion application; not assessed as part of the frontend repository described by this report.

A.3 Required Deliverables Review

Required deliverable	Evidence located	Review status
Source code / GitHub	Client repository on the cover; backend and administration repositories recorded below.	Identified
Frontend application	Client storefront URL recorded in the FED Project.	Recorded
Backend application	Hosted API URL and endpoint catalogue in Sections 6 and A.6.	Recorded
PostgreSQL database	Neon/PostgreSQL is described in the scope and FED planning record.	Verified
API documentation	Section 6 provides methods, endpoints, and frontend usage.	Evidenced
ER diagram	Appendix B reproduces the ER diagram from the FED Project.	Included
Wireframes	Appendix B reproduces five interface wireframes from the FED Project.	Included
Test cases	docs/TEST_CASES.md is referenced in Section 9 and the References list.	Referenced
Bug log	Known warnings and risks are documented in Sections 9 and 11.	Included in Appendix C; all entries remain open pending assignment, correction, retest, and closure approval
Deployment guide	docs/DEPLOYMENT.md is referenced in Sections 10 and References.	Referenced
README	README.md is cited in References.	Referenced
Final project report	This document.	Included
Project presentation	A ten-minute outline is reproduced in A.5.	Outline included

A.4 Five-Day Plan and Recorded Progress

Day	Scope activity	Evidence recorded in the Project
Day 1	Planning, requirements, wireframes, database design, and project setup.	Scope, stories, wireframes, ER diagram, API endpoints, repositories, and the React/Vite scaffold were started or recorded.
Day 2	Backend API development and database integration.	Neon database creation and initial tables were recorded.
Day 3	React frontend development and API integration.	Order/tracking flows, product imagery, and authentication corrections were recorded.
Day 4	Testing, debugging, deployment, and documentation.	Administration work, hosted resource links, and delivery documentation were recorded. Appendix C adds a source-based defect review; full executed-test evidence still requires confirmation.
Day 5	Final validation, presentation, demonstration, and submission.	The final report, presentation outline, and Appendix C bug log are available; the final checklist should be closed with screenshots, retest results, and deployed-flow evidence.

A.5 Working Allocation and Presentation Plan

Workstream	Planned responsibility
Frontend	React components, responsive layouts, shopping experience, cart, authentication screens, localization, and interface integration.
Backend and database	Express endpoints, validation, authentication, PostgreSQL queries, order handling, calculations, and Neon integration.
Quality and delivery	Test execution, bug tracking, deployment support, documentation, screenshot evidence, final report, and presentation preparation.
Shared	Scope decisions, code review, integration testing, demonstration rehearsal, and final submission review.

Time	Presentation section	Content
1 min	Introduction	Project background, problem, and purpose
1 min	Technology	React, Vite, hosted API, Vercel, and the external data service
3 min	Feature demonstration	Products, details, cart, checkout, confirmation, and tracking
2 min	Technical implementation	Frontend structure, API integration, state, and inferred data model
3 min	Testing and conclusion	Results, achievements, limitations, risks, and future work

A.6 Recorded Repositories and Deployments

- **Client source:** [Jade-pang-coder/capstone-project-groupone](#) - storefront repository
- **Backend source:** [shuklasubodh/capstone-project-backend](#) - separate API repository
- **Administration source:** [susutae/project_capstone_admin_react](#) - optional companion administration repository
- **Client storefront:** [capstone-project-groupone.vercel.app](#) - recorded frontend deployment
- **Backend API:** [capstone-project-backend-delta.vercel.app](#) - recorded API deployment
- **Administration interface:** [project-capstone-admin-react.vercel.app](#) - recorded optional admin deployment

A.7 Final Evidence-Closure Checklist

The supplied scope and FED planning record indicate that the following evidence should be closed before final submission:

- Run and record a production smoke test covering the storefront, hosted API, authentication, cart, checkout, and confirmation flow.
- Capture database evidence for representative users, products, orders, and order items without exposing credentials or private customer data.
- Complete the test-result sheet with tester, date, deployed build or commit, expected result, actual result, status, and supporting evidence.
- Assign every Appendix C bug-log entry, record the resolution or accepted deferral, attach retest evidence, and enter the closure date.
- Collect the presentation deck and the screenshots used during the ten-minute demonstration.
- Confirm production URLs, the submitted commit, and each team member's actual contribution before the final team review.

Appendix B - Wireframes and Data Design

The following planning artifacts are reproduced to satisfy the scope requirement for wireframes and an ER diagram. They represent design intent and should not be treated as final implementation screenshots or proof of backend behavior.

B.1 Catalogue and Product Discovery Wireframe

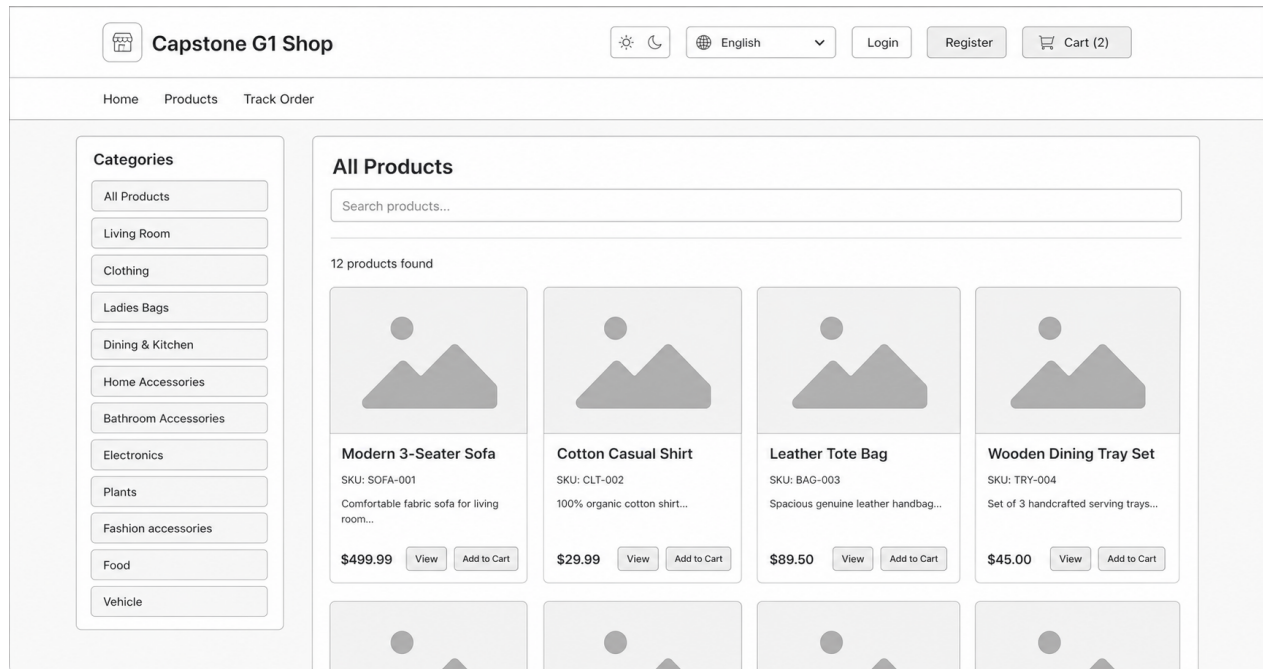


Figure B1. Catalogue wireframe with categories, product search, product cards, language control, account actions, and cart access.

B.2 Member Login Wireframe

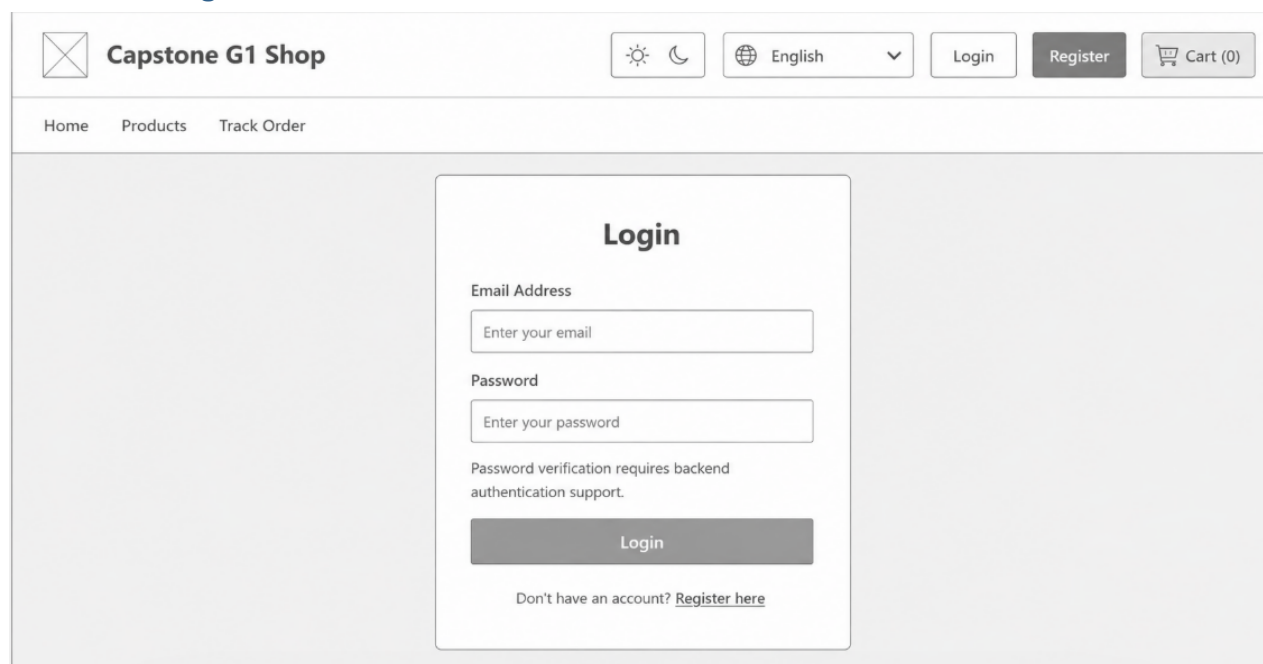
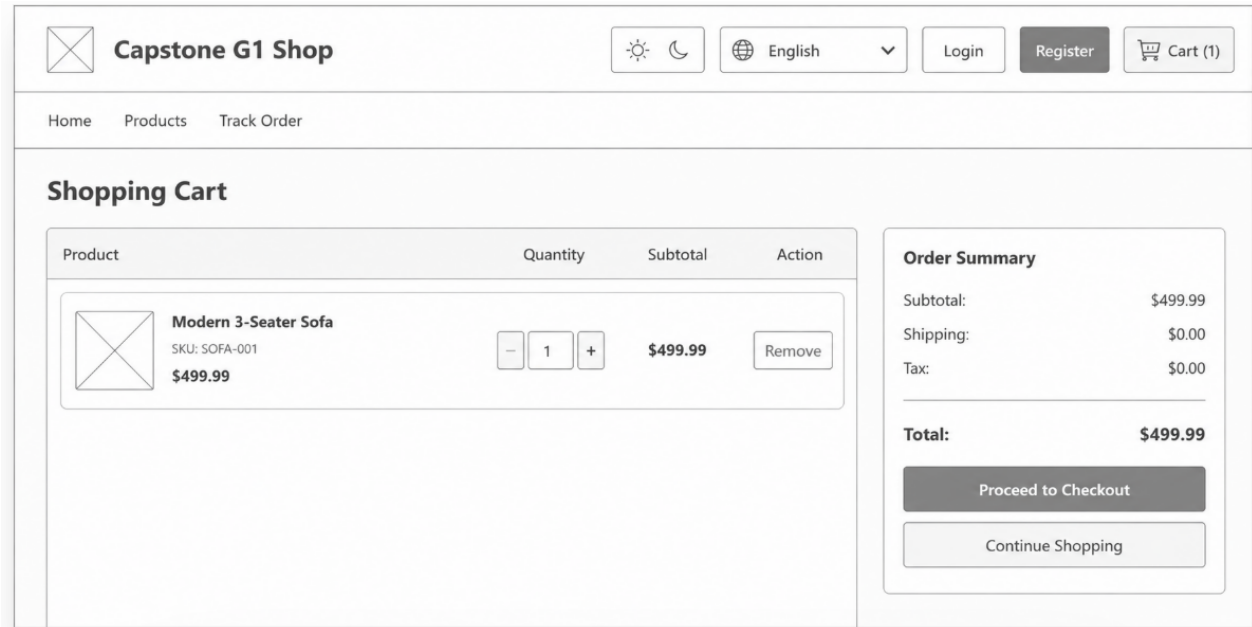


Figure B2. Login wireframe showing email and password fields, validation guidance, and a registration path.

B.3 Shopping Cart Wireframe



The wireframe shows the 'Capstone G1 Shop' header with navigation links (Home, Products, Track Order) and user options (Login, Register, Cart (1)). The main content area is titled 'Shopping Cart' and contains a table of items and an 'Order Summary' sidebar.

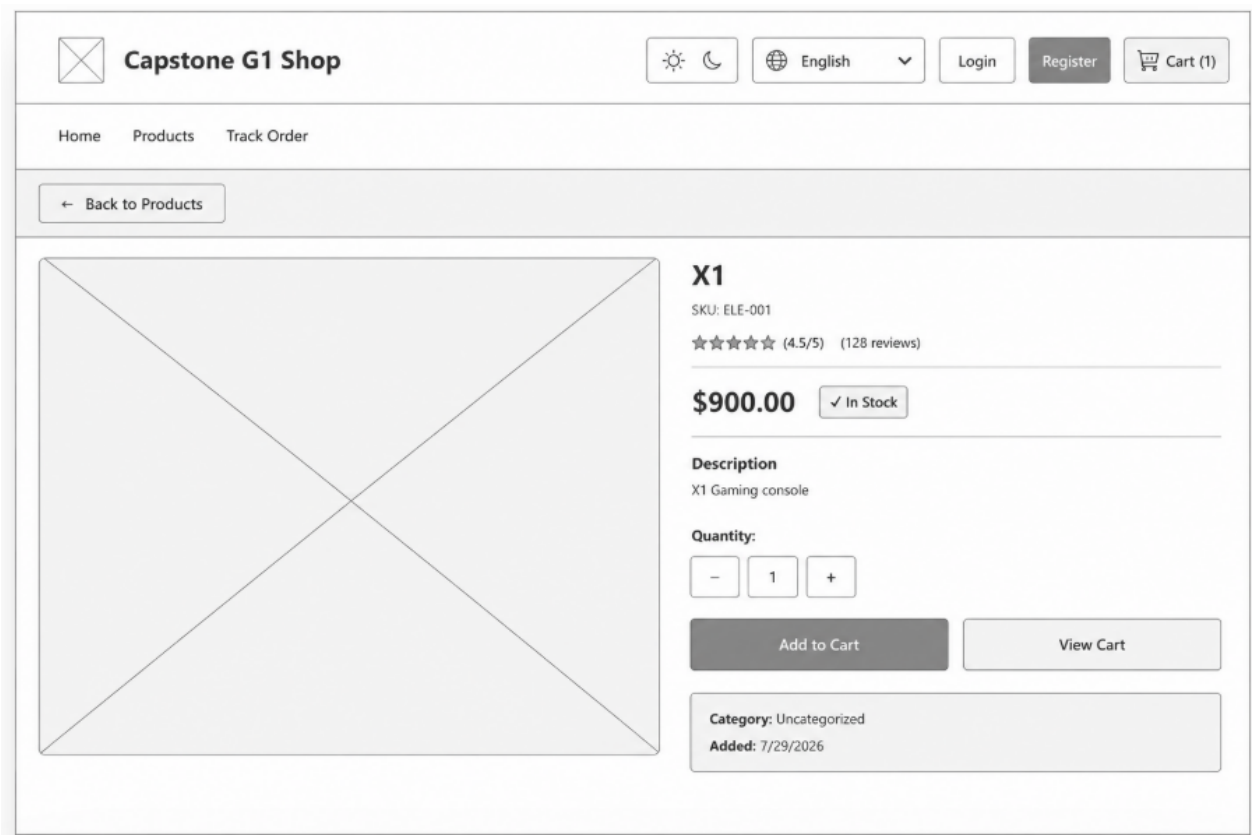
Product	Quantity	Subtotal	Action
 Modern 3-Seater Sofa SKU: SOFA-001 \$499.99	- 1 +	\$499.99	Remove

Order Summary
 Subtotal: \$499.99
 Shipping: \$0.00
 Tax: \$0.00

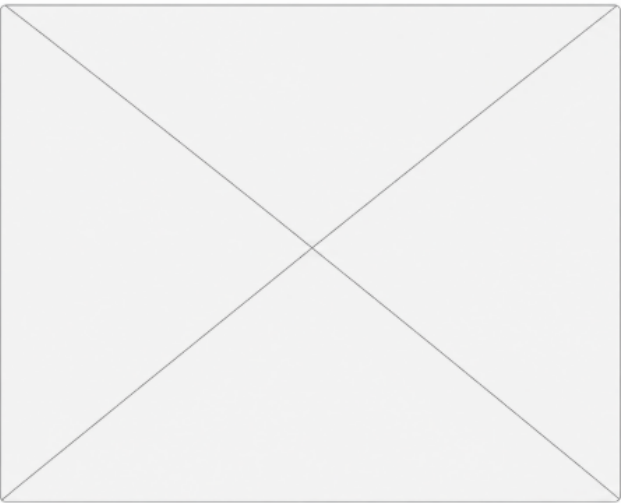
Total: \$499.99

Figure B3. Cart wireframe showing line-item quantity controls, removal, summary totals, and checkout action.

B.4 Product Details Wireframe



The wireframe shows the 'Capstone G1 Shop' header with navigation links (Home, Products, Track Order) and user options (Login, Register, Cart (1)). The main content area features a 'Back to Products' button and a product detail section for 'X1'.



X1

SKU: ELE-001

★★★★☆ (4.5/5) (128 reviews)

\$900.00

Description
X1 Gaming console

Quantity:

Category: Uncategorized
Added: 7/29/2026

Figure B4. Product-detail wireframe with SKU, rating placeholder, price, availability, description, quantity, and cart actions.

B.5 Guest Order Tracking Wireframe

**Capstone G1 Shop**



 English 

Login

Register

 Cart (0)

[Home](#) [Products](#) [Track Order](#)

Track Guest Order

Enter the order number shown on your checkout confirmation.

Order Number

View Order

ORDER NUMBER	STATUS	TOTAL
#ORD-1785331094333-95f6dae4	PENDING	\$499.99

Figure B5. Guest-order tracking wireframe with order-code input and retained order summary.

B.6 Entity Relationship Diagram

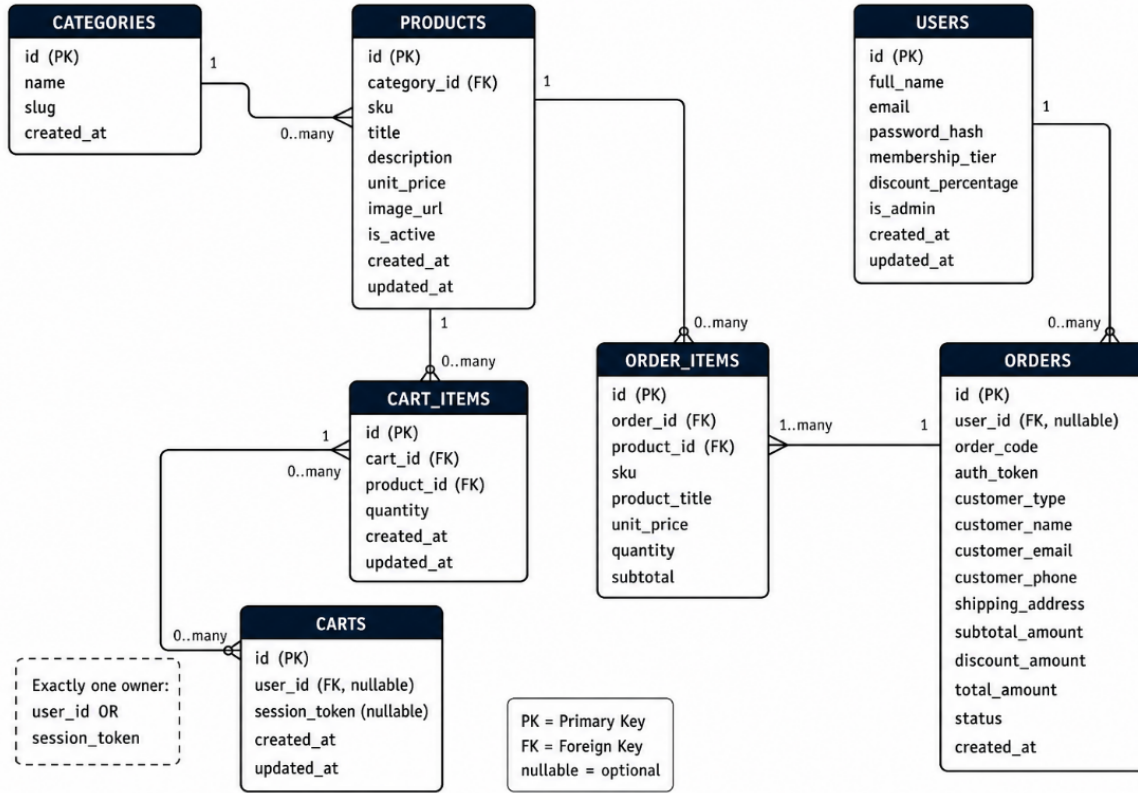


Figure B6. Planning ER diagram for categories, products, users, carts, cart items, orders, and order items. The final backend schema remains outside the reviewed frontend repository.

Appendix C - Source-Based Bug Log

This appendix records defects, security risks, scope variances, and verification gaps identified from the supplied frontend repository. It is a point-in-time engineering review, not proof that every listed risk is exploitable in the hosted environment. Backend authorization, database transactions, and deployed runtime behavior require separate verification.

C.1 Review Basis and Result Summary

- Repository reviewed: capstone-project-groupone, branch doc_update, commit bce2dd5 dated 5 August 2026.
- Production build: PASS on 6 August 2026 using Vite 8.1.5; 92 modules transformed and optimized assets generated.
- Static analysis: PASS with zero errors and four React hook dependency warnings; the warnings are logged as BL-010.
- Automated tests: no test command is defined in package.json. docs/TEST_CASES.md provides manual cases, but an executed result sheet was not supplied.
- Register total: 13 open items - 2 Critical, 4 High, 4 Medium, and 3 Low. Critical and High items should be resolved or formally risk-accepted before production use.

C.2 Bug Log Register

Table C1. Findings, classification, status, and source evidence

ID	Severity	Class	Status	Finding	Evidence
BL-001	Critical	Data integrity	Open	Checkout creates the order first, then creates order items one request at a time. A later failure can leave a header-only or partial order; a retry can create a duplicate.	src/pages/CheckoutPage.jsx:118-125, 155-159
BL-002	Critical	Security	Open	The browser calculates unit prices, discount and totals and sends the monetary values to the order endpoint. Modified client requests could understate the payable total if the API accepts them.	src/pages/CheckoutPage.jsx:75-115; src/api/orderApi.js:87-100
BL-003	High	Security review	Open / verify API	The frontend requests complete order, cart and cart-item collections, then filters records by user or cart in the browser. Confidentiality depends entirely on undocumented server filtering and authorization.	src/api/orderApi.js:38-56; src/api/cartApi.js:10-26; src/context/CartContent.jsx:34-50, 90-94
BL-004	High	Privacy defect	Open	Authenticated cart data is written to the generic localStorage key 'cart'. Logout clears the token and user but not that key, after which guest mode reloads the previous member cart.	src/context/CartContent.jsx:123-136; src/context/AuthContext.jsx:65-71
BL-005	High	Session security	Open	JWT and user data are retained in localStorage and there is no expiry validation or common 401/403 session-reset path. An expired token can leave protected UI state active; an XSS flaw could read the token.	src/context/AuthContext.jsx:7-26, 65-82; docs/TEST_CASES.md TC-AUTH-012
BL-006	High	Data consistency	Open	When some server deletions fail during clear-cart, the UI still replaces the cart with an empty array. The undeleted lines can reappear later and make checkout state misleading.	src/context/CartContent.jsx:289-305
BL-007	Medium	Navigation defect	Open	Navigation exists only in React component state. Refresh resets the application to Home, product/order views have no shareable URL, and browser Back/Forward do not represent page history.	src/App.jsx:16-72
BL-008	Medium	Functional limitation	Open / scope decision	Guest order tracking searches only this browser's localStorage. Clearing site data or using another device loses the order record even when the customer has the order code.	src/pages/GuestOrderLookupPage.jsx:25-48; src/pages/CheckoutPage.jsx:126-153
BL-009	Medium	Test coverage	Open	The repository has a detailed manual test catalogue but no automated test command or recorded execution results. High-risk checkout, authorization and cart transitions lack repeatable regression evidence.	package.json scripts; docs/TEST_CASES.md Sections 4-13

ID	Severity	Class	Status	Finding	Evidence
BL-010	Medium	Code quality	Open	ESLint reports four react-hooks/exhaustive-deps warnings. Effects can retain stale fetch/filter functions after later refactoring and their actual dependencies are not explicit.	DashboardPage.jsx:21; OrderConfirmationPage.jsx:19; ProductDetailsPage.jsx:26; ProductsPage.jsx:26
BL-011	Low	Scope / UX	Open	Checkout displays credit-card, debit-card and PayPal choices, but payment_method is not included in orderData and no payment processing exists. The controls imply behavior outside the documented MVP.	src/pages/CheckoutPage.jsx:18-24, 101-116, 246-284
BL-012	Low	Configuration	Open	The hosted API base URL is repeated as a source-code constant across API modules, making environment changes error-prone and increasing the chance of mixed endpoints.	src/api/authApi.js, cartApi.js, cartItemApi.js, categoryApi.js, orderApi.js, orderItemApi.js, productApi.js
BL-013	Low	UX defect	Open	Footer Privacy, Terms and Contact links point to fragment identifiers for which the application provides no matching sections, so the controls appear interactive but do not open content.	src/App.jsx:79-86

C.3 Corrective Action and Closure Criteria

Table C2. Suggested ownership, corrective action, and objective closure checks

ID	Suggested owner	Corrective action	Retest / closure criteria
BL-001	Backend + frontend	Replace the multi-request flow with one server checkout endpoint and a database transaction. Add an idempotency key and return one complete order response.	An injected item-write failure rolls back the entire order. Repeating the same idempotency key returns the original order and never creates a duplicate.
BL-002	Backend	Accept product identifiers and quantities only. Re-read current prices and eligibility on the server, calculate all totals server-side, and persist immutable line snapshots.	Tampered price, discount and total fields are ignored or rejected, while stored order totals match authoritative product prices and the approved member discount.
BL-003	Backend + security	Provide subject-scoped endpoints such as /users/me/orders and /users/me/cart. Enforce ownership for every read/write on the API and never return another customer's records.	Cross-account API tests return 403/404, guest calls cannot enumerate collections, and responses contain only the authenticated subject's data.
BL-004	Frontend	Clear account cart cache during logout and use separate, user-scoped storage keys. Define and test an explicit guest-to-member cart merge or replacement policy.	After logout, no previous member cart line is visible or present in guest storage. Switching accounts never displays another account's cached cart.
BL-005	Frontend + backend	Implement centralized unauthorized-response handling, expiry checks and a session-expired message. For production, prefer secure HttpOnly/SameSite cookies plus CSRF controls and a restrictive CSP.	Expired/revoked sessions are cleared and redirected consistently; security review confirms the chosen token storage and browser controls.
BL-006	Frontend + backend	Use a server-side clear-cart operation or remove only confirmed lines locally. Treat any deletion failure as incomplete and refresh authoritative cart state before continuing.	A simulated deletion failure leaves the affected line visible with a recoverable error; a successful clear produces an empty server and client cart.
BL-007	Frontend	Adopt React Router with stable routes, guarded account routes and a not-found route. Preserve relevant search and filter state in the URL.	Direct links, refresh, Back/Forward and invalid-route tests pass for catalogue, product, cart, checkout, confirmation, tracking and dashboard views.
BL-008	Product + backend	If cross-device tracking is required, add a rate-limited guest lookup using order code plus a separate verifier. Otherwise label same-browser-only behavior clearly in the interface and acceptance criteria.	The approved behavior is documented and tested: secure server lookup works across browsers, or the UI clearly warns that clearing browser data removes access.
BL-009	QA + development	Add unit/component tests and API-contract tests, then automate critical browser journeys. Record manual test results against the deployed commit with evidence and defect links.	CI runs the automated suite; critical flows pass on the release build; the manual result sheet contains tester, date, environment, result and evidence.
BL-010	Frontend	Stabilize callback dependencies with useCallback or define effect work inside the effect. Re-run lint and exercise the affected fetch/filter paths.	npm run lint reports zero errors and zero warnings; targeted navigation and filtering checks still pass.
BL-011	Product + frontend	For the MVP, replace the controls with a clear 'No online payment in this demo' notice. If payment is later in scope, implement a compliant provider-hosted flow and store only safe payment references.	The interface no longer suggests unimplemented payment processing, or an approved end-to-end payment flow passes acceptance and security testing.

ID	Suggested owner	Corrective action	Retest / closure criteria
BL-012	Frontend / DevOps	Centralize an environment-validated VITE_API_BASE_URL and document local, test and production values. Fail startup/build clearly when required configuration is absent.	All API modules import one validated base URL; test and production builds target their intended endpoints without source edits.
BL-013	Product + frontend	Add approved content/routes and connect the links, or remove the placeholders until the required text is available.	Each footer link opens the intended accessible page/section and passes keyboard plus refresh/deep-link checks.

C.4 Closure Control

All entries are Open because the review did not modify the application or execute the complete manual acceptance catalogue. The assigned owner should add a resolution or approved deferral, link the corrected commit, record the retest result and evidence, and enter a closure date. BL-001 through BL-006 require priority review because they affect order integrity, monetary authority, data access, privacy, session handling, or server/client cart consistency.